TÓPICOS SELECTOS DE ARQUITECTURA DE SOFTWARE · UNIDAD II

Proyecto integrador: producto inteligente en capas con modelos de lenguaje, embeddings y RAG

Descripción del proyecto, requisitos, arquitectura en capas del servicio, entregables y forma de presentación.

Modalidad	Individual.
Forma de presentación	Demostración en vivo, ejecutada de forma local en el equipo del estudiante.
Evaluación	Rúbrica de evaluación del proyecto (documento anexo). Producto: 60 puntos. Informe técnico: 40 puntos. Total: 100 puntos.
Fecha de entrega	**7 de noviembre de 2026 (07/11/2026)**: repositorio, informe técnico y presentación.

1. El producto: más allá de un chatbot

Cada estudiante **imaginará**, diseñará, construirá y presentará un **producto inteligente** para un dominio y un tipo de usuario elegidos por él mismo. El producto usará **modelos de lenguaje ejecutados localmente**, representará su conocimiento mediante **embeddings** guardados en una base vectorial y fundamentará sus resultados con **generación aumentada por recuperación (RAG)**.

El producto no puede ser un chatbot. Una ventana de preguntas y respuestas es la forma más obvia de usar un modelo, y por eso no basta. El reto creativo es encontrar una forma nueva de poner los modelos al servicio de alguien: que analicen, comparen, generen, recomienden, clasifiquen, extraigan, transformen o vigilen, y que entreguen un **resultado útil** (un informe, un borrador, una recomendación, una alerta, una tabla, un mapa) dentro del flujo de trabajo del usuario. Puede existir una conversación, pero solo como función secundaria.

El servicio que implementa la lógica de negocio debe seguir una **arquitectura en capas** e incluir, de forma obligatoria, las capas de una aplicación de IA en producción: construcción de contexto, guardrails, enrutador y gateway de modelos, caché, orquestación con patrones de agente, observabilidad y retroalimentación del usuario (sección 4). El producto se entrega como un **servicio que expone una API** y se usa desde una **aplicación web**.

> **Idea clave:** se califica la creatividad de la idea tanto como su ejecución. La calificación tiene dos partes: el **producto** (60 puntos, de los cuales 10 son por creatividad y propuesta de valor) y el **informe técnico** (40 puntos), que demuestra que el estudiante diseñó la arquitectura y no solo la programó.

1.1 Prueba rápida: ¿es un chatbot?

Si se quita la caja de chat y el producto ya no aporta nada, todavía es un chatbot. Un buen producto responde con claridad estas tres preguntas:

1. **¿Quién lo usa y en qué momento de su trabajo?** Una persona concreta con una tarea concreta, no «cualquier usuario».
2. **¿Qué resultado entrega?** Un artefacto o una acción (un documento revisado, una lista priorizada, una alerta, un borrador), no solo un mensaje de texto.
3. **¿Qué hace el modelo que antes requería mucho tiempo o un experto?** Ese es el valor que debe poder medirse.

1.2 Catálogo de ideas

Los siguientes tipos de producto son punto de partida, no una lista cerrada. Se valora especialmente combinar dos tipos o adaptarlos a un problema local (la universidad, la comunidad, un negocio familiar, la salud, la agricultura, el turismo, las lenguas originarias).

Tipo de producto	Qué hacen los modelos	Papel de embeddings y RAG	Ejemplo
Auditor de documentos	Revisa un documento contra normas o criterios y señala hallazgos con su gravedad.	Recupera la norma aplicable a cada sección del documento.	Revisor de tesis contra el reglamento de titulación y la guía de estilo; revisor de contratos de arrendamiento contra la ley local.
Generador fundamentado	Redacta borradores a partir de plantillas y fuentes, citando de dónde sale cada parte.	Recupera ejemplos, plantillas y datos de apoyo.	Generador de planes de clase alineados al programa de la materia; generador de ADR a partir de notas de una reunión.
Emparejador o recomendador semántico	Relaciona personas, recursos o proyectos y explica por qué.	Compara perfiles y descripciones por similitud de embeddings.	Emparejar estudiantes con asesores de tesis o proyectos de servicio social; recetas con ingredientes de temporada de la región.
Explorador visual de conocimiento	Agrupa, resume y nombra temas; detecta huecos.	Agrupamiento (clustering) y proyección de embeddings en un mapa.	Mapa de las tesis de la universidad que muestra temas saturados y temas poco explorados.
Clasificador y priorizador	Clasifica solicitudes, asigna prioridad y propone una respuesta.	Recupera casos parecidos y la política que aplica.	Triaje de solicitudes de servicios escolares o de reportes ciudadanos.
Detector de similitud	Encuentra duplicados, plagio semántico o propuestas parecidas.	Búsqueda de vecinos cercanos en la base vectorial.	Detector de propuestas de proyecto parecidas a las de generaciones anteriores.
Tutor y generador de evaluaciones	Crea ejercicios, califica respuestas abiertas con rúbrica y da retroalimentación.	Recupera el material de la clase para que las preguntas y la calificación se apoyen en él.	Banco de preguntas generado desde los apuntes, con retroalimentación personalizada.
Transformador multimodal	Convierte audio o imágenes en documentos estructurados.	Indexa transcripciones o descripciones de imágenes para buscarlas después.	De grabaciones de reuniones a minutas con acuerdos y tareas; de fotos de etiquetas a fichas de producto.
Extractor de datos	Pasa documentos sin estructura a tablas o JSON consultables.	Recupera los fragmentos donde está cada dato.	Extraer requisitos y fechas de convocatorias de becas y armar un calendario personalizado.

Simulador o juego de rol	Interpreta un personaje con base en fuentes reales y evalúa el desempeño del usuario.	Recupera el caso y los criterios de evaluación.	Simulador de entrevistas de trabajo basado en vacantes reales; paciente simulado para estudiantes de enfermería.
Monitor y generador de alertas	Revisa fuentes periódicas y avisa solo de lo relevante, con una explicación.	Compara lo nuevo contra el perfil de interés del usuario.	Alertas para productores de café sobre precios, clima y plagas a partir de boletines.
Verificador de cumplimiento	Recorre una lista de requisitos y emite un reporte de cumplimiento.	Recupera el requisito y la evidencia correspondiente.	Verificador de accesibilidad de páginas web; revisión de expedientes contra una lista de requisitos.

1.3 Cómo encontrar una idea original

- **Empezar por una persona y una tarea, no por la tecnología.** Observar a alguien (un docente, un comerciante, un trabajador de salud) y anotar qué tarea repetitiva o difícil hace con documentos o información.
- **Exagerar:** ¿qué haría un experto si tuviera tiempo ilimitado para esa persona? El producto hace una parte de eso.
- **Combinar dos tipos del catálogo:** por ejemplo, un extractor de datos con un monitor de alertas, o un auditor con un generador que propone la corrección.
- **Cambiar el resultado:** en lugar de responder, que el producto entregue una tabla, un mapa, una línea de tiempo, una lista priorizada o un documento listo para firmar.
- **Cambiar la entrada:** en lugar de una pregunta escrita, que reciba un archivo, una foto, un audio, un formulario o un evento.
- **Pensar en lo local:** problemas de la región, de la universidad o del entorno del estudiante suelen dar ideas más originales y con usuarios reales para probar.

1.4 Ficha de la idea

En la etapa 1 cada estudiante entrega esta ficha para que el docente apruebe la idea antes de construirla:

Elemento	Respuesta del estudiante
Nombre del producto	
Usuario y contexto de uso	
Problema que resuelve	
Resultado que entrega (no un chat)	
Tipo de producto (del catálogo o combinación)	
Cómo intervienen los modelos, los embeddings y el RAG	

Qué lo hace original	
Cómo se medirá su valor	

2. Objetivos de aprendizaje

Al terminar el proyecto, el estudiante será capaz de:

1. Proponer un producto original que integre modelos de lenguaje en el flujo de trabajo de un usuario concreto, más allá de una conversación.
2. Diseñar un servicio basado en modelos de lenguaje con una arquitectura en capas, con responsabilidades y dependencias bien definidas.
3. Incorporar las capas que necesita una aplicación de IA en producción: contexto, guardrails, enrutamiento, caché, orquestación, observabilidad y retroalimentación.
4. Seleccionar modelos de lenguaje y de embeddings con criterios explícitos y evidencia experimental.
5. Construir un flujo RAG completo: ingesta, fragmentación, embeddings, indexado, recuperación y generación con fuentes.
6. Exponer el servicio mediante una API documentada, consumirla desde una aplicación web y evaluar la calidad del sistema con métricas.

3. Requisitos obligatorios

Todos los requisitos de esta tabla son obligatorios. Si falta alguno de los marcados con (*), el proyecto no puede obtener calificación aprobatoria, sin importar el puntaje del resto (ver la rúbrica).

ID	Requisito	Cómo se verifica
R1 (*)	**Dominio definido por el estudiante.** Elegir un dominio de problema real, describir a sus usuarios y reunir un corpus propio (mínimo sugerido: 20 documentos o 50 páginas).	Sección de dominio en el informe; corpus en el repositorio o descrito con su origen y licencia.
R2 (*)	**Ejecución local.** Todo el sistema corre en el equipo del estudiante: modelos servidos localmente (por ejemplo, con Ollama), base vectorial, servicio y aplicación web. No se usan APIs de modelos en la nube.	Demostración en vivo en el equipo del estudiante, idealmente sin conexión a internet.
R3 (*)	**Selección de modelos sustentada.** Comparar al menos 2 modelos de lenguaje y 2 modelos de embeddings con criterios explícitos y evidencia medida.	Tabla comparativa y conclusión en el informe; scripts del experimento.
R4 (*)	**Embeddings.** Generar embeddings del corpus y guardarlos en una base vectorial (Chroma, FAISS, Qdrant, pgvector u otra).	Código de ingesta e indexado; endpoint de ingesta funcionando.
R5 (*)	**RAG.** Recuperar los fragmentos relevantes y usarlos como contexto del modelo. Cada respuesta indica sus fuentes.	Respuestas de la demostración con fuentes citadas.

R6 (*)	**Arquitectura en capas del servicio.** La lógica de negocio se organiza en capas e incluye todas las capas obligatorias de la sección 4.	Estructura del código, diagrama de capas y demostración de cada capa.
R7 (*)	**Servicio con API.** La funcionalidad se expone como un servicio con API REST documentada (por ejemplo, OpenAPI con FastAPI).	Documentación de la API accesible; pruebas de los endpoints.
R8 (*)	**Capa de presentación web.** Una aplicación web consume la API por HTTP. No llama directamente a los modelos ni a la base vectorial.	Revisión del código y del diagrama de arquitectura.
R9	**Evaluación.** Medir la calidad del sistema y de cada capa con conjuntos de prueba (sección 9).	Resultados y análisis en el informe.
R10	**Reproducibilidad.** Repositorio con instrucciones para instalar y ejecutar todo (README, dependencias, modelos requeridos).	El docente puede levantar el sistema siguiendo el README.
R11 (*)	**Informe técnico.** Documento con la arquitectura, los diagramas de secuencia, los ADR, la selección de modelos y la evaluación (sección 11).	Entrega del informe en la fecha indicada.
R12 (*)	**Producto más allá de un chatbot.** La función principal entrega un resultado (análisis, borrador, recomendación, alerta, tabla, mapa) dentro del flujo de trabajo del usuario. Una conversación solo puede ser función secundaria.	Ficha de la idea aprobada; demostración.

4. Arquitectura en capas del servicio

El sistema tiene dos contenedores: la **aplicación web** (capa de presentación) y el **servicio**, que implementa la lógica de negocio. Dentro del servicio, el código se organiza en capas. La regla es la de cualquier arquitectura en capas: **cada capa solo usa a las capas que están debajo de ella**, a través de interfaces claras, y ninguna capa inferior conoce a las superiores. Las dos capas transversales (observabilidad y retroalimentación) pueden ser usadas por todas.

4.1 Capas obligatorias

Capa	Responsabilidad	Requisito mínimo
C1. Interfaz del servicio (API)	Recibe las peticiones HTTP, valida los datos de entrada y salida y traduce cada petición en una llamada a la capa de orquestación. No contiene lógica de IA.	Endpoints de la sección 6; validación con esquemas (por ejemplo, Pydantic); códigos de error adecuados.
C2. Orquestación y patrones de agente	Coordina el flujo de cada consulta a través de las capas inferiores. Implementa el comportamiento de agente: ciclos de decisión y uso de herramientas.	Flujo explícito y legible; al menos un ciclo de agente (por ejemplo, si la recuperación es pobre, reformular la pregunta y volver a buscar, con un máximo de iteraciones).

C3. Guardrails (entrada y salida)	Protege al sistema y al usuario. En la entrada: datos personales, intentos de inyección de instrucciones, preguntas fuera del dominio. En la salida: formato, citas de fuentes, datos sensibles, apego al contexto.	Al menos 2 verificaciones de entrada y 2 de salida, con una política definida para cada falla (bloquear, reintentar o responder con un mensaje seguro).
C4. Caché	Evita trabajo repetido. Caché exacta para preguntas idénticas y caché semántica para preguntas parecidas, comparando sus embeddings.	Ambos tipos de caché, con umbral de similitud justificado, tiempo de expiración e invalidación cuando cambian los documentos.
C5. Construcción de contexto	Reúne la información que necesita el modelo: recuperación de fragmentos (RAG), herramientas opcionales y armado del prompt.	RAG con fuentes; parámetros de fragmentación, k y umbral justificados.
C6. Enrutador y gateway de modelos	El enrutador clasifica cada consulta (por intención o complejidad) y elige el modelo adecuado. El gateway ofrece una interfaz única hacia todos los modelos, con tiempo límite, reintentos, modelo de respaldo y registro de uso.	Al menos 2 modelos de lenguaje enrutados con un criterio explícito; gateway con tiempo límite y modelo de respaldo.
C7. Acceso a datos y modelos	Adaptadores hacia los recursos: servidor de modelos (Ollama), base vectorial, almacén de documentos, caché, retroalimentación y bitácoras.	Un adaptador por recurso; cambiar de base vectorial no debe obligar a modificar las capas superiores.
T1. Observabilidad y monitoreo (transversal)	Registra lo que pasa en cada capa para depurar, medir y mejorar.	Identificador de traza por consulta; bitácoras estructuradas por capa; métricas de latencia por capa, aciertos de caché, activaciones de guardrails, modelo elegido y retroalimentación.
T2. Retroalimentación del usuario (transversal)	Recoge la opinión del usuario sobre cada respuesta y la liga con su traza.	Retroalimentación explícita (útil / no útil y comentario) guardada y usada en el análisis de la evaluación.

En este documento, **consulta** significa cualquier petición que procesa el servicio: analizar un documento, generar un borrador, recomendar, clasificar o responder.

4.2 Flujo de una consulta

1. La aplicación web envía la pregunta a la **API (C1)**, que valida la petición.
2. La **orquestación (C2)** crea un identificador de traza y conduce el resto del flujo.
3. Los **guardrails de entrada (C3)** revisan la pregunta. Si falla una verificación, se responde con un mensaje seguro.
4. La **caché (C4)** busca una respuesta exacta o semánticamente parecida. Si la encuentra, se salta al paso 8.
5. El **enrutador (C6)** clasifica la consulta y elige el modelo.
6. La **construcción de contexto (C5)** recupera los fragmentos relevantes. Si la recuperación es pobre, el **ciclo de agente (C2)** reformula la pregunta y vuelve a buscar.
7. El **gateway (C6)** llama al modelo elegido a través del **acceso a modelos (C7)**, con tiempo límite y respaldo.

8. Los **guardrails de salida (C3)** revisan la respuesta. Si falla una verificación, se reintenta o se responde con un mensaje seguro.
9. Se guarda la respuesta en la caché, se registran las métricas (**T1**) y la API devuelve la respuesta con sus fuentes y su identificador de traza.
10. El usuario califica la respuesta desde la aplicación web (**T2**).

4.3 Estructura sugerida del código

```
proyecto/
  web/                    # Capa de presentación (aplicación web)
  servicio/
    api/                  # C1  Interfaz del servicio
    orquestacion/         # C2  Flujo de consulta y ciclo de agente
    guardrails/           # C3  Verificaciones de entrada y salida
    cache/                # C4  Caché exacta y semántica
    contexto/             # C5  RAG, herramientas y armado del prompt
    modelos/              # C6  Enrutador y gateway
    datos/                # C7  Adaptadores: Ollama, base vectorial, almacenes
    observabilidad/       # T1  Bitácoras, trazas y métricas
    retroalimentacion/    # T2  Registro de la opinión del usuario
  evaluacion/             # Conjuntos de prueba y scripts de evaluación
  docs/                   # Diagramas y ADR
```

> **Precaución:** si la aplicación web se hace con Streamlit o Gradio, la tentación es llamar al modelo directamente desde la interfaz. Eso incumple R8. La interfaz debe llamar a la API como cualquier otro cliente.

4.4 Tecnologías sugeridas

Elemento	Opciones
Aplicación web	HTML + JavaScript, React, Vue, Svelte; Streamlit o Gradio si consumen la API por HTTP.
Servicio y API	FastAPI (recomendado) o Flask.
Modelos de lenguaje	Ollama con llama3.2, qwen2.5, mistral o gemma (versiones que quepan en el equipo).
Modelos de embeddings	nomic-embed-text, mxbai-embed-large, bge-m3.
Base vectorial	ChromaDB, FAISS, Qdrant, pgvector.
Caché	Diccionario en memoria, SQLite o Redis.
Observabilidad	Bitácoras en JSON con el módulo logging; métricas en SQLite o Prometheus; panel en la aplicación web.

5. Selección de modelos

La elección de los modelos es una decisión de arquitectura y debe sustentarse con evidencia. Se compararán **al menos 2 modelos de lenguaje** y **al menos 2 modelos de embeddings**, con el mismo corpus y las mismas preguntas de prueba. El resultado alimenta directamente al enrutador (C6): por

ejemplo, un modelo pequeño y rápido para preguntas simples y uno más capaz para preguntas complejas.

Criterio	Qué se mide o se revisa
Calidad de las respuestas	Exactitud y apego al contexto en las preguntas de prueba.
Calidad de la recuperación	Para embeddings: si los fragmentos correctos aparecen entre los k primeros (acierto@k).
Latencia	Tiempo promedio de respuesta en el equipo del estudiante.
Recursos	Memoria RAM o VRAM requerida, tamaño en disco, cuantización usada.
Idioma	Desempeño en español, si el corpus está en español.
Ventana de contexto	Cuántos fragmentos caben en el prompt.
Licencia	Si permite el uso previsto (académico, comercial).

Formato sugerido para la tabla comparativa:

Modelo	Parámetros / cuantización	Calidad (0–5)	Latencia (s)	Memoria (GB)	Licencia
llama3.2:3b					
qwen2.5:7b					
(otro)					

La conclusión explica qué modelo se usa para cada tipo de consulta, qué se sacrificó y por qué ese compromiso es adecuado para el dominio y el equipo disponible.

6. API mínima del servicio

El servicio debe ofrecer, como mínimo, los siguientes endpoints. Los nombres pueden cambiar, pero la funcionalidad no.

Método y ruta	Propósito	Respuesta esperada
GET /health	Comprobar que el servicio, los modelos, la base vectorial y la caché están disponibles.	Estado de cada componente.
POST /documentos	Cargar documentos; el servicio los fragmenta, genera embeddings e indexa.	Identificador del documento y número de fragmentos.
GET /documentos	Listar los documentos indexados.	Lista con identificador, nombre y fecha.
DELETE /documentos/{id}	Eliminar un documento, sus fragmentos y las entradas de caché afectadas.	Confirmación.

POST /{funcion-principal}	La función principal del producto (por ejemplo, POST /analisis, /borradores, /recomendaciones, /clasificaciones).	Resultado, fuentes, modelo usado, uso de caché, guardrails activados, tiempo e identificador de traza.
POST /retroalimentacion	Registrar la opinión del usuario sobre una respuesta.	Confirmación.
GET /metricas	Consultar las métricas de observabilidad.	Latencias por capa, aciertos de caché, activaciones de guardrails, uso por modelo, retroalimentación.
GET /configuracion	Mostrar los modelos y parámetros activos.	Configuración actual.

Ejemplo para un auditor de tesis, **POST /analisis**:

```
// Petición
{ "documento_id": "tesis-0425", "criterios": "reglamento_titulacion" }

// Respuesta
{
  "resumen": "3 hallazgos: 1 grave, 2 menores",
  "hallazgos": [
    { "seccion": "Portada", "gravedad": "grave",
      "descripcion": "Falta el nombre del director de tesis",
      "fuente": { "documento": "reglamento.pdf", "articulo": "42", "puntaje": 0.86 } }
  ],
  "modelo": "qwen2.5:7b",
  "ruta": "analisis_completo",
  "cache": "ninguna",
  "guardrails": { "entrada": "aprobada", "salida": "aprobada" },
  "iteraciones_agente": 2,
  "tiempo_ms": 8450,
  "traza_id": "c7f2a1"
}
```

7. Flujo RAG: decisiones que se deben documentar

Etapa	Decisión	Ejemplos de opciones
Ingesta	Formatos aceptados y limpieza del texto.	PDF, DOCX, HTML, Markdown; quitar encabezados y pies de página.
Fragmentación	Tamaño del fragmento y solapamiento.	500 a 1,000 caracteres con 10–20 % de solapamiento; por párrafos o por secciones.
Embeddings	Modelo y normalización.	nomic-embed-text, mxbai-embed-large, bge-m3.
Indexado	Base vectorial y metadatos guardados.	Metadatos de documento, página y sección.
Recuperación	Número de fragmentos (k), métrica de similitud y umbral.	k entre 3 y 8; similitud coseno; descartar resultados bajo un umbral.
Generación	Prompt del sistema y manejo de «no sé».	Responder solo con el contexto; citar fuentes; decir cuando no hay información.

8. Registro de decisiones de arquitectura

Cada decisión importante se registra como un **ADR** (registro de decisión de arquitectura). Se esperan **al menos 6 ADR**, entre ellos: selección de modelos, criterio del enrutador, umbral de la caché semántica, verificaciones de los guardrails, fragmentación y diseño del ciclo de agente. Los ADR forman parte del informe técnico.

Plantilla de ADR:

```
ADR-03: Umbral de similitud de la caché semántica
Estado:        Aceptada
Contexto:      Qué problema o necesidad obliga a decidir; qué atributo de calidad está en
juego.
Decisión:      Qué se decidió, en una o dos frases.
Alternativas:  Qué otras opciones se consideraron y por qué se descartaron.
Evidencia:     Datos medidos que respaldan la decisión (tabla, gráfica, prueba).
Consecuencias: Qué se gana, qué se sacrifica y qué riesgos quedan.
```

9. Evaluación del sistema

El estudiante construirá conjuntos de prueba de su dominio y medirá tanto la calidad de las respuestas como el funcionamiento de cada capa:

Qué se evalúa	Conjunto de prueba	Métricas
Resultados del producto (RAG)	Al menos 20 casos de prueba (entradas) con el resultado esperado y su fuente.	Acierto@k de la recuperación; fidelidad al contexto; relevancia (0–5); comparación con y sin RAG.
Guardrails	Al menos 10 entradas problemáticas (inyección de instrucciones, fuera de dominio, datos personales) y 10 entradas normales.	Porcentaje de bloqueos correctos y de falsos positivos.
Caché	Preguntas repetidas y parafraseadas.	Tasa de aciertos; reducción de latencia; respuestas incorrectas servidas desde la caché semántica.
Enrutador	Preguntas etiquetadas por tipo.	Porcentaje de consultas enviadas al modelo correcto; costo en latencia de cada ruta.
Ciclo de agente	Preguntas con recuperación inicial pobre.	Porcentaje que mejora tras reformular; iteraciones promedio.
Desempeño	Todo lo anterior.	Latencia promedio y máxima, total y por capa (a partir de las trazas).

Se permite usar un modelo como juez para calificar fidelidad y relevancia, siempre que se explique el procedimiento y se revise una muestra a mano. La retroalimentación registrada durante las pruebas se incluye en el análisis.

10. Entregables

Entregable	Contenido
Repositorio de código	Servicio organizado por capas, aplicación web, scripts de ingesta y de evaluación; README con instalación y ejecución; archivo de dependencias; lista de modelos que hay que descargar.
Informe técnico (PDF)	Contenido de la sección 11. Extensión sugerida: 15 a 25 páginas. Vale 40 de los 100 puntos.
Documentación de la API	OpenAPI generada por el servicio, más ejemplos de uso.
Conjuntos de prueba y resultados	Archivos CSV o JSON con las pruebas y sus resultados.
Presentación	Diapositivas de apoyo y demostración en vivo (sección 12).

11. Informe técnico (40 puntos)

El informe técnico es una parte central del proyecto: vale **40 de los 100 puntos** y es obligatorio (R11). En él el estudiante demuestra que **diseñó** la arquitectura: qué decidió, por qué, con qué evidencia y cómo se comportan las capas en tiempo de ejecución. El código muestra que el sistema funciona; el informe muestra que está bien pensado.

> **Importante:** sin informe técnico el proyecto no puede aprobar, aunque el producto funcione. Los diagramas y los ADR deben corresponder a lo que realmente está implementado.

11.1 Estructura

1. **Portada y resumen** (media página).
2. **La idea del producto:** usuario, problema, resultado que entrega, propuesta de valor y qué lo hace original (ficha de la idea).
3. **Dominio del problema:** contexto y descripción del corpus (origen, tamaño, licencia).
4. **Requisitos y atributos de calidad:** funcionales y no funcionales priorizados y medibles (latencia, precisión, seguridad, privacidad, uso de recursos).
5. **Arquitectura:** vistas estructurales (ver 11.2), con la responsabilidad e interfaz de cada capa del servicio.
6. **Comportamiento:** diagramas de secuencia de los escenarios principales (ver 11.2).
7. **Selección de modelos y enrutamiento:** experimento, tabla comparativa, justificación y criterio del enrutador.
8. **Diseño de cada capa:** construcción de contexto, guardrails, caché, gateway, ciclo de agente, observabilidad y retroalimentación.
9. **Registro de decisiones (ADR):** al menos 6, con la plantilla de la sección 8.
10. **Evaluación:** conjuntos de prueba, métricas, resultados en tablas o gráficas y análisis de errores.
11. **Limitaciones, riesgos y trabajo futuro:** alucinaciones, sesgos, privacidad y seguridad.
12. **Referencias** y anexo con el README y la documentación de la API.

11.2 Diagramas obligatorios

Diagrama	Qué debe mostrar
Contexto	El sistema, sus usuarios y los sistemas externos con los que se relaciona.
Contenedores	Aplicación web, servicio, servidor de modelos (Ollama), base vectorial y almacenes, con los protocolos entre ellos.
Capas del servicio	Las capas C1 a C7 y las transversales T1 y T2, con la dirección de sus dependencias.
Componentes	Los módulos principales de cada capa y sus interfaces.
Despliegue local	Qué procesos corren en el equipo del estudiante, en qué puertos y con qué recursos.
Secuencia: ingesta	Carga de un documento: fragmentación, embeddings, indexado e invalidación de la caché.
Secuencia: consulta normal	Recorrido completo por las capas, del paso 1 al 10 de la sección 4.2.
Secuencia: consulta desde caché	Una pregunta parecida a otra anterior que se responde desde la caché semántica.
Secuencia: consulta bloqueada	Una entrada que los guardrails rechazan y la respuesta segura que recibe el usuario.
Secuencia: ciclo de agente	Una consulta con recuperación pobre que el agente reformula y vuelve a buscar.

Notación sugerida: modelo C4 para las vistas estructurales y UML para los diagramas de secuencia y de despliegue. Herramientas sugeridas: PlantUML, Mermaid, Structurizr o draw.io. Todos los diagramas llevan número, título y una breve explicación.

12. Presentación local

La presentación se hace en vivo, en el equipo del estudiante, con todos los componentes ejecutándose localmente. Duración sugerida: **15 minutos de exposición y 5 de preguntas**.

Tiempo	Contenido
2 min	La idea: usuario, problema, resultado que entrega y por qué es original.
3 min	Arquitectura: contenedores, capas del servicio y flujo de una consulta.
3 min	Selección de modelos, criterio del enrutador y decisiones de RAG, con su evidencia.
5 min	Demostración: cargar un documento; consulta con fuentes; la misma pregunta servida desde la caché; una entrada bloqueada por los guardrails; una consulta que activa el ciclo de agente; registrar retroalimentación; panel de métricas; la API en su documentación OpenAPI.
2 min	Resultados de la evaluación y limitaciones.

Calidad de la presentación. Además de la demostración, se evalúa la exposición: estructura clara, manejo del tiempo, diapositivas legibles apoyadas en diagramas (no en párrafos), vocabulario técnico preciso y seguridad al responder preguntas sobre las decisiones de arquitectura.

> **Recomendación:** descargar los modelos y preparar el índice antes de la sesión. Tener un plan B (por ejemplo, un video corto de la demostración) por si falla el equipo, aunque la evaluación se hace sobre la demostración en vivo.

13. Etapas de trabajo sugeridas

Todas las etapas deben estar terminadas para la **fecha de entrega: 7 de noviembre de 2026**. Se recomienda escribir el informe a la par del código, no al final.

Etapa	Trabajo	Evidencia
1	Idea, corpus y diseño de capas	Ficha de la idea aprobada por el docente; corpus; diagramas de contexto, contenedores y capas; esqueleto del servicio con la API.
2	Datos, contexto y gateway	Adaptadores de datos y modelos; ingesta, embeddings y recuperación; gateway; experimento de embeddings; primeros ADR.
3	Enrutador, guardrails, caché y orquestación	Experimento de modelos y enrutador; guardrails; caché exacta y semántica; ciclo de agente; aplicación web; diagramas de secuencia.
4	Observabilidad, evaluación e informe	Trazas y métricas; retroalimentación; conjuntos de prueba evaluados; informe técnico completo; ensayo de la presentación.

14. Lineamientos éticos y académicos

- Usar solo documentos que se tenga derecho a usar y citar su origen y licencia.
- No incluir datos personales o sensibles en el corpus; si el dominio los requiere, anonimizarlos.
- Declarar en el informe el uso de asistentes de IA para programar o redactar, y en qué partes se usaron.
- El sistema debe advertir al usuario que las respuestas son generadas por un modelo y pueden contener errores.
- El trabajo es individual y original. Se puede reutilizar código de ejemplo citando su fuente.

15. Resumen de la evaluación

La calificación se asigna con la rúbrica anexa y tiene dos partes:

Criterio	Puntos
Parte A. Producto	**60**

A1. Creatividad, originalidad y propuesta de valor (R12)	10
A2. Construcción de contexto: embeddings y RAG (C5, R4, R5)	8
A3. Arquitectura en capas del servicio (R6)	8
A4. Guardrails de entrada y salida (C3)	5
A5. Enrutador, gateway y caché (C4, C6)	6
A6. Orquestación y patrones de agente (C2)	4
A7. Observabilidad y retroalimentación (T1, T2)	5
A8. API y aplicación web (C1, R7, R8)	7
A9. Demostración local en vivo (R2)	3
A10. Calidad de la presentación	4
Parte B. Informe técnico	**40**
B1. Idea, dominio, requisitos y atributos de calidad	4
B2. Arquitectura: vistas y diagramas estructurales	8
B3. Diagramas de secuencia	6
B4. Registro de decisiones de arquitectura (ADR)	7
B5. Selección de modelos y enrutamiento (R3)	6
B6. Evaluación y análisis de resultados (R9)	6
B7. Redacción, formato y reproducibilidad (R10)	3
Total	**100**